

Spider In The Pod

—— 零权限突破 k8s 集群的初始入口

演讲者: Esonhugh



🤔：离开了 RBAC 和传统容器逃逸，
Kubernetes 集群是不是就无法攻破？

假设这样一个背景，你攻破了一个业务的容器，现在你拥有了一个集群 Pod 的 SHELL，你尝试了多种技巧来尝试突破，包括

- 服务账户的 RBAC 利用
- 特殊特权位 Linux CAP
- 各种未授权 kubelet、dockerd、apiserver
- 内核利用
- 云服务厂商 Metadata

.....

但是很可惜以上的方法，都以失败告终。

遇到低权容器的原因

- 仅仅将 k8s 作为容器编排系统，调度处理业务容器启停，而不利用 k8s 内部各种机制进行业务管理或业务之间相互管理
- 高价值容器主要工作在集群视角的控制面上，而非数据面，且不暴露在外
- 低/无权限业务容器常常直接暴露于外网，且相对较易被攻破
- 对安全的重视，做了良好的最小化 RBAC 权限处理

Pod 可以做什么



阿里云



阿里云先知
Alibaba Cloud XianZhi



Alibaba Security
Resource Center
阿里巴巴安全资源中心

- Pod 内自我网络权限
- 任意其他 Pod 网络权限和 Service 访问能力
- 集群网络内节点宿主机的访问能力

传统内网信息收集的局限性

- k8s 网段较为复杂
 - Service 所在网段
 - Pod 所在网段
 - 宿主机所在网段
 - 云服务网段 VPC Metadata
- 存在 network sidecar 的 k8s pod

Introducing - k8spider

为了对抗这些复杂情况，我根据自己实际渗透过程中寻找到的脆弱点和利用手段，开发了k8spider。

工具地址：<https://github.com/Esonhugh/k8spider>



目录 Contents



通用信息泄漏



DNS



Metrics



NFS



Mutating
Webhook

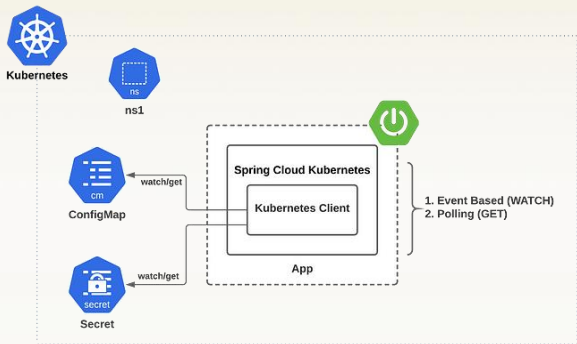


Validating
Webhook

通用信息泄漏与 k8s 结合

传统信息采集在 k8s 中可能会有奇效。以 Java SpringCloud Kubernetes 和 Java Spring actuator heapdump 为例，说明集群内 Java Application 的一个较为普遍问题。

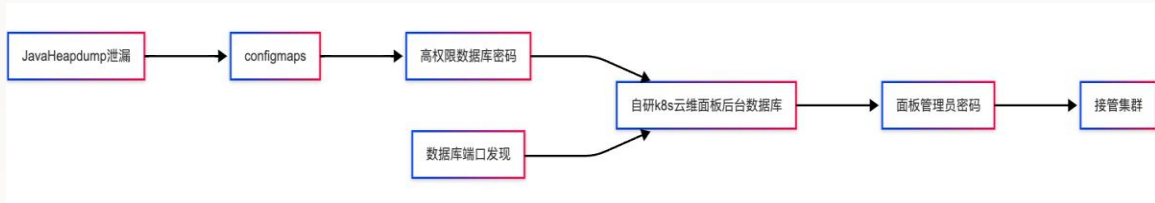
在 Java SpringCloud Kubernetes 中会使用 K8S 的 configmap 或 secrets 作为配置中心，那么在官方文档的模式下，默认会配置出一个具有 ConfigMap 或 Secrets 读权限的服务账户。当和 actuator 组合在一起使用时，黑客就可以拿到一个可以访问 APISERVER 的有效服务账户 Token



案例一 Heapdump 集群失陷

在未授权情况下获取 heapdump 后，分析出 fabric8 kubernetes client 和 okhttp client 中存放的 k8s service account token。然后通过这个 token 直接访问 apiserver，于是直接获取到了集群命名空间下的所有配置信息。

在审计配置的过程中，恰好发现存有一个中央数据库地址的配置，而数据库又被 NodePort 进行暴露到了外部端口。通过访问这个数据库，截获到了 dashboard 的 admin 后台密码，最后登陆，接管了集群。



K8S 的 DNS 协议默认就存在多种服务发现的方式。

其主要是以下两种

1. wildcard dns query 实现
2. 多播 DNS 的 DNS-SD 实现

Wildcard DNS 也叫通配符 DNS 解析，本质原理是使用 * 作为通配符解析 DNS。

在早期 k8s dns 中，我们可以使用诸如 `*.*.svc.cluster.local` 的 SRV 记录，来获取整个集群的所有 Service 信息。（此处 * 可以使用 any 替换）但是由于这个利用点广为人知，且经常被黑客利用，导致 CoreDNS 在 1.9.0 版本中删除了这项支持。

MDNS 中 DNS-SD 协定，是在局域网中联合使用 PTR、SRV、TXT 三项记录来进行服务查找。

- **PTR (Pointer Record)**：将服务类型映射到具体实例名。
- **SRV (Service Record)**：定义服务实例的地址和端口。
- **TXT (Text Record)**：携带服务实例的元数据（如版本、协议参数）。

而在 K8S 中，他们的 DNS 协议经过了一些改变，做了一些特殊的兼容。例如，PTR 查询 Pod IP 或 Service IP 会得到对应的 Service 暴露的名称，再通过 Service 的名称直接查询 SRV 则会直接获取到该 service 下的所有端口信息。

因此，通过遍历 POD CIDR 以及 SERVICE CIDR 我们便可以获取通过上述三种 DNS 请求获取整个集群的所有服务名称以及对应开放的端口。

与 Wildcard dns 解析类似, Axfr 则是 DNS Zone Transfer 的一种传输协议, 这项技术本身不是用于**服务发现**的, 而是用于区域间 DNS 服务器记录转移和同步的。当然这一项技术也没有鉴权策略, 主要靠 acl 控制块控制。而且 Coredns 支持通过配置如下的 transfer 配置块, 来配置出存在任意 Pod Axfr 请求 CoreDNS 的能力。

黑客可以利用这一点, 获取整个集群的 dns 记录。

```
Corefile: |
  .:53 {
    errors
    health
    kubernetes cluster.local in-addr.arpa ip6.arpa {
      pods insecure
      fallthrough in-addr.arpa ip6.arpa
    }
    prometheus :9153
    forward . /etc/resolv.conf
    cache 30
    loop
    reload
    loadbalance
    transfer {
      to *
    }
  }
```

K8Spider 中核心支持的能力，便是服务发现。通过上述提到的三类基于 DNS 的服务利用，只需要如下一条命令就可以自动进行枚举。
Spider 会自动发现环境中周围的服务地址和 pod ip 进行枚举，并且输出结果。

Displaying logs from Namespace: devcontainer for Pod: k8spider. Logs from 2025/4/13 GMT+8 14:22:14

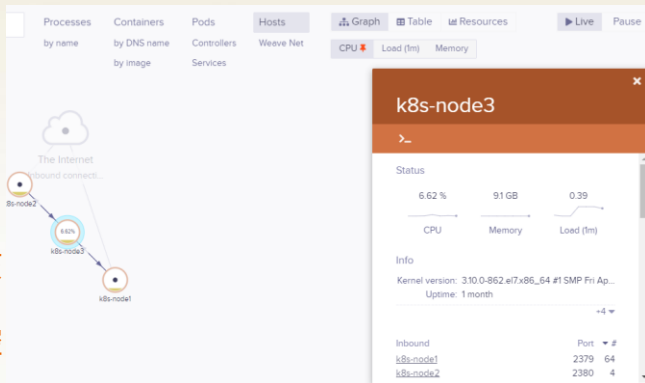
```
time="2025-04-13T06:22:14Z" level=warning msg="wildcard dns query to any.any.svc.cluster.local. failed: lookup any.any.svc.cluster.local: no such host"
time="2025-04-13T06:22:14Z" level=warning msg="wildcard dns query to any.any.any.svc.cluster.local. failed: lookup any.any.any.svc.cluster.local: no such host"
time="2025-04-13T06:22:14Z" level=error msg="Transfer failed: dns: bad xfr rcode: 5"
time="2025-04-13T06:22:14Z" level=info msg="PTRRecord 10.43.0.1 --> kubernetes.default.svc.cluster.local."
time="2025-04-13T06:22:14Z" level=info msg="SRVRecord: kubernetes.default.svc.cluster.local. --> kubernetes.default.svc.cluster.local."
time="2025-04-13T06:22:14Z" level=info msg="PTRRecord 10.43.0.10 --> kube-dns.kube-system.svc.cluster.local."
time="2025-04-13T06:22:14Z" level=info msg="SRVRecord: kube-dns.kube-system.svc.cluster.local. --> kube-dns.kube-system.svc.cluster.local."
time="2025-04-13T06:22:14Z" level=info msg="SRVRecord: kube-dns.kube-system.svc.cluster.local. --> kube-dns.kube-system.svc.cluster.local."
time="2025-04-13T06:22:14Z" level=info msg="SRVRecord: 10-0-0-18.esoncloud-demo-svc.default.svc.cluster.local. --> 10-0-0-18.esoncloud-demo-svc.default.svc.cluster.local."
time="2025-04-13T06:22:14Z" level=info msg="SRVRecord: 10-0-0-25.cert-manager-webhook.cert-manager.svc.cluster.local. --> 10-0-0-25.cert-manager-webhook.cert-manager.svc.cluster.local."
time="2025-04-13T06:22:14Z" level=info msg="SRVRecord: 10-0-0-36.hubble-relay.kube-system.svc.cluster.local. --> 10-0-0-36.hubble-relay.kube-system.svc.cluster.local."
time="2025-04-13T06:22:14Z" level=info msg="SRVRecord: 10-0-0-42.app.hackmd.svc.cluster.local. --> 10-0-0-42.app.hackmd.svc.cluster.local."
time="2025-04-13T06:22:14Z" level=info msg="SRVRecord: 10-0-0-54.cert-manager.cert-manager.svc.cluster.local. --> 10-0-0-54.cert-manager.cert-manager.svc.cluster.local."
```

案例二 Weave Anonymous Admin

Weave Scope 是一个很有名的 k8s dashboard。但是默认情况不需要鉴权，仅仅只开放在集群内网，使用 Service 暴露。

他的 DNS 名称通常是 `weave-scope-app.weave.svc.cluster.local`。

如果你在服务发现的过程中发现了该服务或者类似的名称，访问其端口，便可以看到管理员面板，进而直接控制整个集群。



监控指标 (Metrics) 也是云上很常见的服务，对运维和可视化而言，至关重要。

而且 metrics 类服务通常不需要授权而且对集群范围内开放，敏感信息虽然不是很多但是这通常需要具体情况具体判断。

而其中，我们最为关心的应该是集群类指标。而在这其中最为有代表性的是 [kube-state-metrics](#)，这是 k8s 官方的 metrics 服务。他会采集集群中工作负载，配置，网络，甚至是角色绑定等信息，并且将其暴露在打开的 8080 端口的 /metrics 路由下。

但是 metrics 的信息较为杂乱，分析起来容易遗漏内容。于是根据 metrics 文本我在 k8spider 中研发了解析 kube-state-metrics 的能力。

```
root ➔ /workspace $ wget http://kube-state-metrics.lens-metrics.svc.cluster.local:8080/metrics
--2025-04-13 06:58:57-- http://kube-state-metrics.lens-metrics.svc.cluster.local:8080/metrics
Resolving kube-state-metrics.lens-metrics.svc.cluster.local (kube-state-metrics.lens-metrics.svc.cluster.local)... 10.43.222.53
Connecting to kube-state-metrics.lens-metrics.svc.cluster.local (kube-state-metrics.lens-metrics.svc.cluster.local)|10.43.222.53|:8080... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/plain]
Saving to: 'metrics'

metrics                                     [ <=> ] 412.08K --.-KB/s   in 0.002s

2025-04-13 06:58:57 (173 MB/s) - 'metrics' saved [421971]

root ➔ /workspace $ ./k8spider metrics metrics
{"namespace":"kube-system","type":"configmap","name":"kube-root-ca.crt","spec":{}}
{"namespace":"localstack","type":"configmap","name":"kube-root-ca.crt","spec":{}}
{"namespace":"blood","type":"configmap","name":"bloodhound-config","spec":{}}
{"namespace":"blood","type":"configmap","name":"kube-root-ca.crt","spec":{}}
{"namespace":"default","type":"configmap","name":"nginx-config","spec":{}}
{"namespace":"istio-system","type":"configmap","name":"istio-gateway-status-leader","spec":{}}
{"namespace":"kube-system","type":"configmap","name":"cilium-config","spec":{}}
{"namespace":"kube-system","type":"configmap","name":"local-path-config","spec":{}}
{"namespace":"ingress-nginx","type":"configmap","name":"nginx-ingress-leader","spec":{}}
{"namespace":"kube-system","type":"configmap","name":"hubble-relay-config","spec":{}}
{"namespace":"discord","type":"configmap","name":"discorder-config","spec":{}}
{"namespace":"localstack","type":"configmap","name":"istio-ca-root-cert","spec":{}}
```

1. 与 **ConfigMaps** , **Secrets** 相关的指标, **命名空间和名称**
2. 与节点相关的指标, 内核版本、操作系统镜像、容器运行时版本、提供商ID和内部IP 节点权限 (是否为 mater)
3. 与 Pods 相关的指标, 命名空间、Pod 名称、节点、主机 IP 和 Pod IP
4. 与容器, 初始化容器相关的指标, 命名空间、Pod、容器、镜像规格和镜像
5. 与服务账户相关的指标, 命名空间、挂载的 Pod 和服务账户
6. 与服务相关的指标, 命名空间、服务、集群 IP、外部名称、端点地址 IP 列表和端口号列表
7. 与持久卷相关的指标, 添加存储类、不同提供商的磁盘名称、**NFS服务器和路径**、CSI驱动程序和卷句柄、本地路径和文件系统、主机路径和类型等各种属性
8. 匹配与 Webhook 相关的指标, 例如 **mutating webhook** 或者 **validating webhook**, 与对应的服务位置信息。

NFS 模式是 k8s 服务常见的持久化卷模式。包括 AWS 阿里云均具有此类服务，可以帮助集群持久化数据存储。

不仅可以尝试横向其他目录的数据文件也可以尝试一些潜在的配置存放。

而且在一些数据库持久化中，NFS 持久卷可能会被用于存储例如 MongoDB data 目录、MySQL Data 目录等等。

通过直接访问这些敏感的数据库 DATA 文件，我们可以很轻易的获取到一些数据库本身的账户凭证或是数据库存放的内容。

NFS 理论上最低权限只需要网络可达即可访问，但是一般通过挂载实现访问。如果 shell 的权限较低或无法联网使用 apt 等包管理下载 mount.nfs nfsclient 时，就较难进行利用。

于是我在 k8spider 中开发了第三个功能 NFS client。

```
I have no name!@mongodb-1:/tmp$ ./k8spider nfs -s $AP.cn-hangzhou.nas.aliyuncs.com ls
.
..
data
I have no name!@mongodb-1:/tmp$ ./k8spider nfs -s $AP.cn-hangzhou.nas.aliyuncs.com ls -a
drwxrwxrwx    0      0      4096   2023-11-23 13:51:49  .
drwxrwxrwx    0      0      4096   2023-11-23 13:51:49  ..
drwxr-xr-x    0      0      4096   2025-04-12 17:56:06  data
```

PostgreSQL，是一个十分流行的数据库。有些管理员也会使用他来作为集群的中心化的数据库存储。

但是数据库上 k8s 的问题在于需要持久化的数据存储，于是管理员往往会选择使用 PV/PVC 进行数据托管。而当集群变得越来越大，节点数量变多，使用 HostPath 类型的 PV 在数据管理上代价变大。于是很多管理员会选择一开始就使用 NFS 类型的 PV。

当我们在 k8s 中暴露的 NFS 里发现了 PostgreSQL 的 DATA 目录时，对我们而言相当于是提供了一个在 PostgreSQL 中有限的任意文件读+写原语。无论是配置文件修改，上传恶意 so，还是读取敏感数据文件、密码等，都可以让我们在集群中继续横向移动。

MutatingWebhook 是 k8s 提供的一项扩展机制，它主要用于在资源对象（如 Pod、Deployment 等）被创建或修改时，APIServer 会主动向管理员配置的 MutatingWebhook 发送 AdmissionReview 申请，从而根据 Webhook 的返回，**动态地修改其配置**。，允许开发者通过自定义逻辑干预资源对象的生命周期。

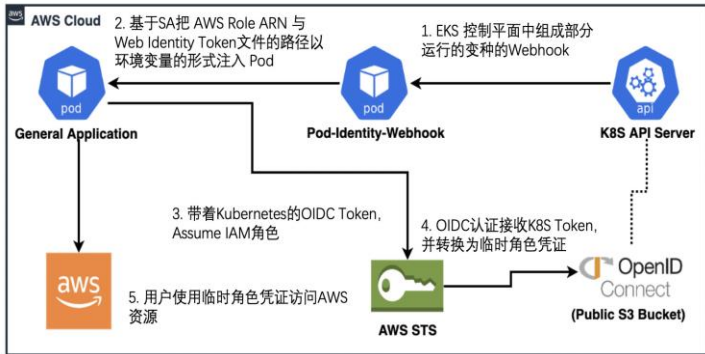
黑客较为关心的是他针对一些环境变量注入配置的能力。例如，当集群配置了 mutating webhook 来注入敏感配置到环境变量，并且没有强制验证请求来源来自 APIServer 的时候，我们可以直接伪造一个 Admission Review 资源来请求 webhook 欺骗对方将敏感配置文件注入到 Review 结果中。

案例四 - Mutating AKSK 注入

有一些云原生开发者会使用这项机制做动态的 AccessKey 管理，或者说 AK Rotation。这样做可以有效地提高一部分 AK 的安全性。这样业务就不需要操心 AK 具体是什么，也不用担心将 AK 的配置复制的到处都是。甚至是泄漏后，也可以第一时间轮换掉。

但是如果一些开发者如果贪图便利，将明文凭证直接注入到环境变量里，这样我们就可以通过伪造 Pod 创建过程，欺骗

Mutating Webhook 注入配置到返回中。



ValidatingWebhook 也是一项 k8s 提供的扩展机制，它主要用于在资源对象（如 Pod、Service、Ingress 等）被创建或修改时，**验证其合法性**。它通过调用外部 Webhook 服务对资源进行校验，确保其符合集群的安全、合规或业务规则。

集群会在你创建某些资源或配置的时候通过 Validating Webhook 进行检查，从而确认参数是否填写正确，是否有需要修改的地方和缺少的字段。但是一旦存在某些注入促使我们可以控制服务器上部分内容，则会导致一些较为特殊的安全问题。

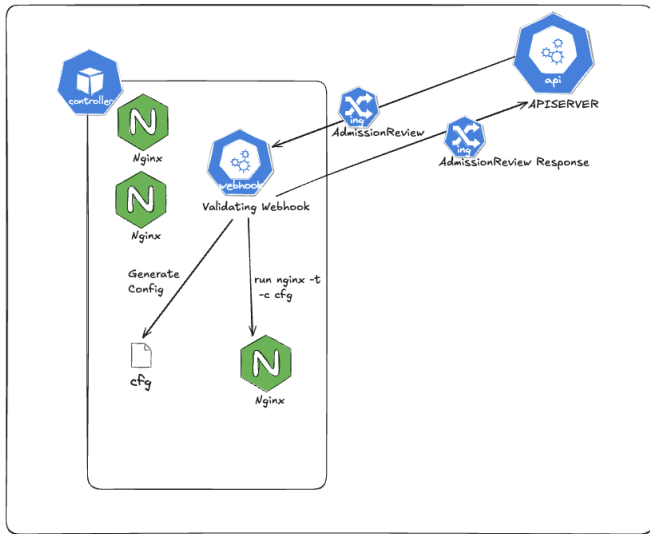
案例五 - IngressNightmare

今年爆发的 CVE-2025-1974 IngressNightmare 由云安全独角兽公司 Wiz 的研究团队发现。



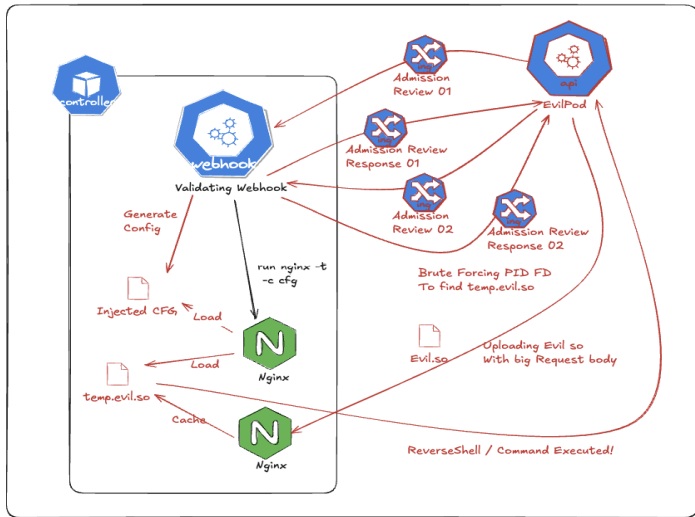
案例五 - IngressNightmare

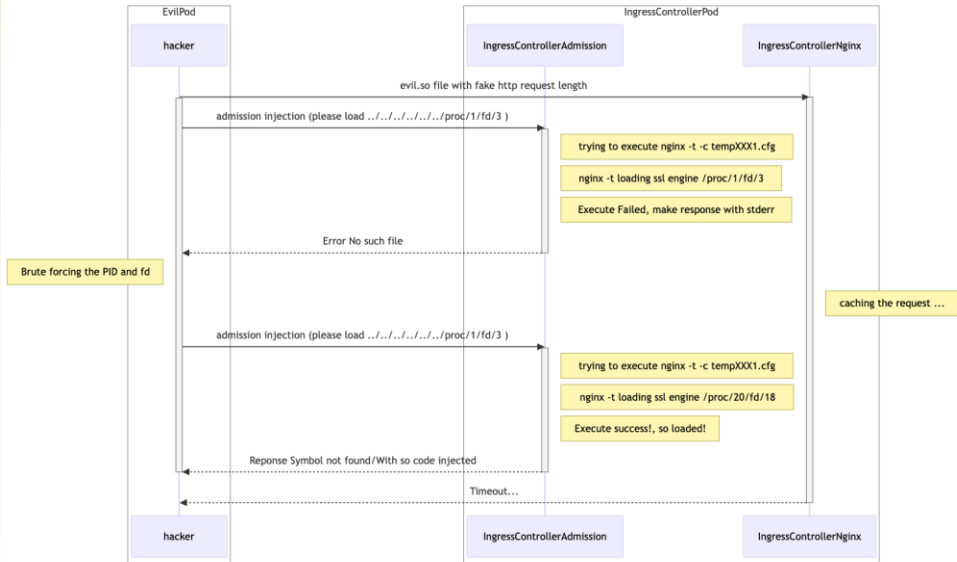
正常情况下的流程中，
当 Ingress 资源被修改时，
APIServer 像 Webhook 发起 Admission Review。
Validating Webhook 收到后，
检查参数并且生成临时
文件配置 cfg，然后创建进
程 nginx -t 来测试整个配置
是否正常。
最后根据 nginx -t 的结果来
返回 AdmissionReview 的
Response



案例五 - IngressNightmare

而黑客则是生成恶意的 so 文件然后通过 nginx 缓存机制将文件缓存到 webhook 所在的 pod 中，同时发送恶意 Admission Review 注入恶意的 nginx 配置，使得 admission webhook 在测试配置的时候创建的 nginx -t 进程加载缓存的 so 文件。最后完成命令执行。整个过程完全不涉及鉴权





About me.

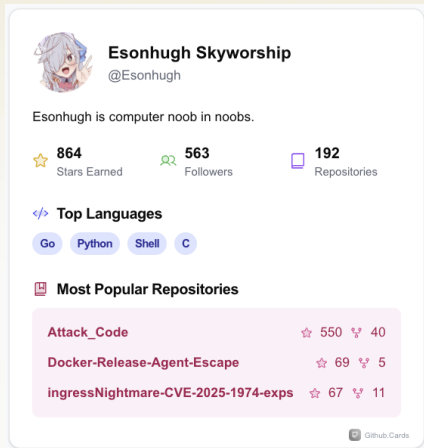


如果你对我的工作感兴趣的话，欢迎关注我的Github。

<https://github.com/Esonhugh/>

我的其他云安全相关研究内容，可以在下面的仓库中找到。

<https://github.com/Esonhugh/My-Cloud-Security/>



Esonhugh Skyworship
@Esonhugh

Esonhugh is computer noob in noobs.

★ **864** Stars Earned 👥 **563** Followers 📁 **192** Repositories

🔗 **Top Languages**

Go Python Shell C

📁 **Most Popular Repositories**

Attack_Code	★ 550	👤 40
Docker-Release-Agent-Escape	★ 69	👤 5
ingressNightmare-CVE-2025-1974-exps	★ 67	👤 11

GitHub Cards

问答环节

Q&A

Thanks !

